

머신러닝 11 교시

— 차원축소 dimension reduction —

차원 축소 개요(Overview)

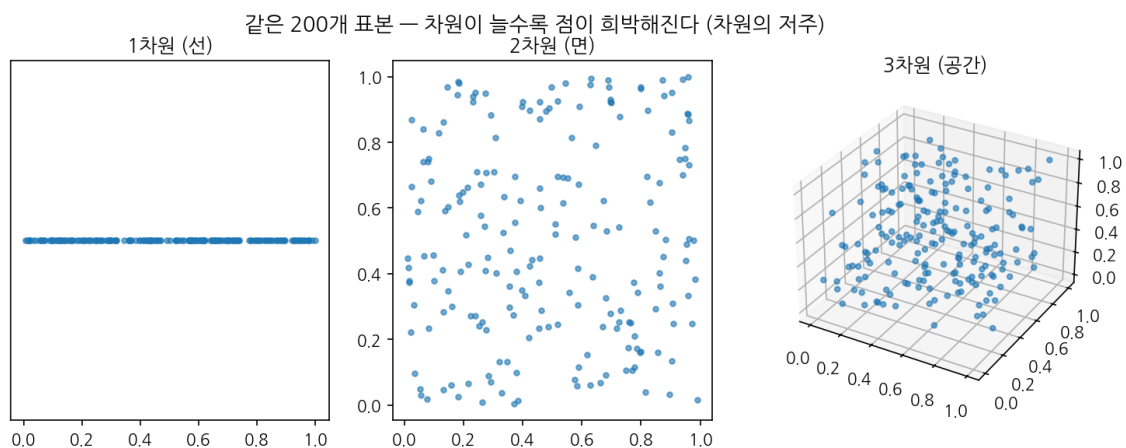
차원 축소란 무엇이고 왜 필요한가

데이터를 다루다 보면 특징(feature)이 수십, 수백, 때로는 수만 개에 이르는 경우가 흔하다. 유전자 발현 데이터는 변수가 2만 개를 넘고, 한 장의 28×28 이미지조차 784 개의 픽셀 값을 가진다. 옛날에는 '데이터가 많으면 많을수록 좋다'고 단순하게 생각했지만, 막상 변수가 늘어나면 모델이 더 똑똑해지기는커녕 오히려 길을 잃는 일이 벌어진다. 차원 축소는 바로 이 문제, 즉 '너무 많아서 탈인' 변수를 줄여 데이터의 본질만 남기려는 시도다.

차원 축소는 원래의 고차원 데이터를, 중요한 정보는 최대한 보존하면서 더 적은 수의 차원으로 다시 표현하는 작업이다. 100 개의 변수를 2~3 개로 줄이면 사람이 눈으로 볼 수 있게 되고(시각화), 변수 사이의 중복과 잡음이 정리되며, 모델 학습에 드는 계산 비용도 크게 줄어든다.

차원 축소가 필요한 네 가지 이유 — ① 차원의 저주 완화: 차원이 늘수록 데이터가 희박해져 거리·밀도 기반 알고리즘이 무력해진다. ② 시각화: 사람은 2~3 차원까지만 직관적으로 본다. 고차원을 2D 로 내려야 눈으로 구조를 파악할 수 있다. ③ 노이즈 제거와 중복 정리: 서로 상관 높은 변수들을 묶어 잡음을 걸러낸다. ④ 계산 비용 절감과 과적합 완화: 변수가 적으면 학습이 빠르고, 불필요한 자유도가 줄어 일반화에 유리하다.

특히 '차원의 저주(Curse of Dimensionality)'는 차원 축소를 공부할 때 반드시 짚고 넘어가야 하는 개념이다. 차원이 하나 늘어날 때마다 그 공간을 채우는 데 필요한 데이터의 양은 기하급수적으로 증가한다. 아래 그림처럼 같은 200 개의 표본이라도 1 차원에서는 뽕뽕하지만, 3 차원으로 가면 점들이 서로 멀찍이 떨어져 '텅 빈 공간'이 대부분을 차지한다. 점 사이 거리가 모두 비슷해지면서 '가까운 이웃'이라는 개념 자체가 흐려진다.



[그림 0-1] 차원의 저주 — 같은 표본 수라도 차원이 늘면 공간이 희박해진다

2차원 축소의 큰 분류

차원 축소 기법은 크게 두 축으로 나뉘 볼 수 있다. 첫째는 '데이터를 어떻게 변환하는가'에 따른 분류이고, 둘째는 '레이블(정답)을 쓰는가'에 따른 분류다. 이 두 축을 머릿속에 그려 두면 새로운 기법을 만나도 금방 자리를 잡아 줄 수 있다.

선형 기법 vs 비선형 기법

차원 축소 큰 그림 - 선형(Linear): 데이터를 곧은 평면·직선 축에 투영한다. 빠르고 해석이 쉽고 새 데이터에 그대로 적용 가능. 예) PCA, LDA. 비선형(Non-linear, 매니폴드 학습): 휘어진 곡면(매니폴드) 위에 데이터가 놓여 있다고 보고 그 곡면을 펼친다. 복잡한 구조를 잘 잡지만 보통 시각화 전용이고 새 데이터 예측은 어렵다. 예) t-SNE, UMAP, Isomap.

선형 기법은 '원본 변수들의 가중합'으로 새 축을 만든다. 직선·평면이라 수식이 명료하고, 한번 학습한 변환을 새로운 표본에도 똑같이 적용할 수 있다. 반면 비선형 기법은 데이터가 휘어진 곡면 위에 분포한다고 가정한다. 유명한 '스위스 롤(swiss roll)' 데이터처럼 둘둘 말린 구조는 직선 투영으로는 절대 펼 수 없고, 비선형 기법이라야 원래의 평평한 구조를 복원한다.

지도학습 vs 비지도학습 관점

또 다른 분류 축은 레이블 사용 여부다. PCA와 t-SNE는 정답 레이블 없이 데이터의 분포만 보고 축을 찾는 비지도(unsupervised) 기법이다. 반면 LDA는 각 표본이 어느 클래스인지를 알아야 동작하는 지도(supervised) 기법이다. LDA는 '클래스를 잘 가르는 방향'을 찾기 때문에, 분류 성능을 끌어올리려는 목적이라면 PCA보다 유리할 때가 많다.

세 알고리즘 비교

이 교재에서 다룰 세 알고리즘 PCA, LDA, t-SNE를 한 표로 정리하면 다음과 같다. 지금은 표의 각 칸이 낯설어도 괜찮다. 각 절을 읽고 다시 이 표로 돌아오면 모든 칸이 또렷하게 이해될 것이다.

구분	PCA	LDA	t-SNE
선형/비선형	선형	선형	비선형(매니폴드)
지도/비지도	비지도	지도(레이블 필요)	비지도
핵심 목표	분산 최대 보존	클래스 분리 최대화	이웃 관계 보존
주 용도	압축·노이즈제거·전처리	분류 전 차원 축소	시각화(2D/3D)
최대 축 개수	원본 차원까지	클래스수 - 1	보통 2~3
새 데이터 변환	가능(transform)	가능(transform)	어려움(재학습)
대표 활용	이미지 압축, EDA	얼굴 인식 전처리	군집 구조 탐색

표에서 가장 눈여겨볼 칸은 맨 아래 '새 데이터 변환'이다. PCA와 LDA는 학습으로 얻은 변환 규칙(축)을 저장해 두었다가 처음 보는 데이터에도 그대로 적용할 수 있다.

1. 선형 차원 축소 (Linear Methods)

1.1 PCA — 주성분 분석 (Principal Component Analysis)

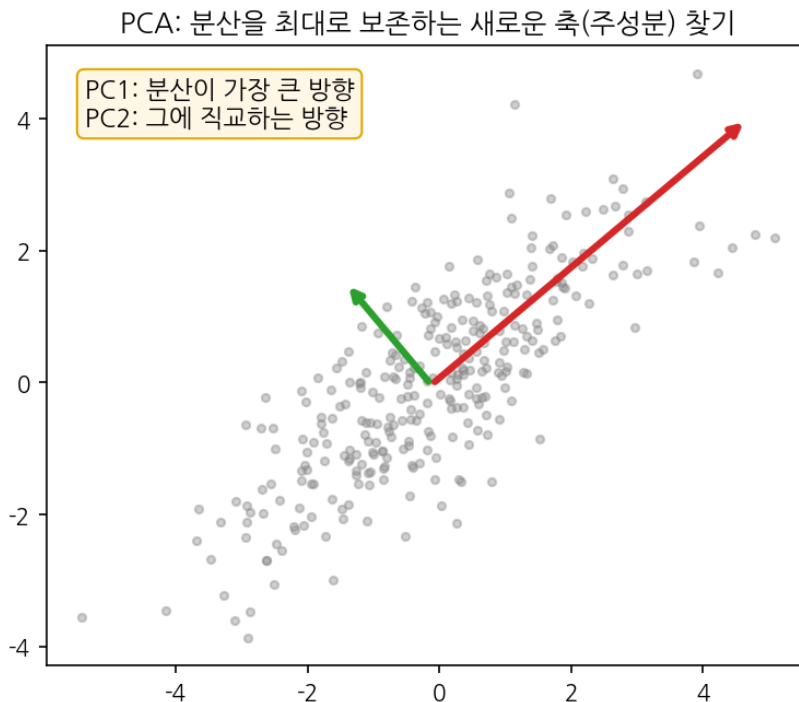
차원 축소를 처음 배울 때 거의 모든 사람이 가장 먼저 만나는 기법이 PCA 다. 1901 년 칼 피어슨이 처음 제안한 이 방법은 100 년이 넘도록 데이터 압축과 탐색의 표준 도구로 살아남았다. 과거에는 변수가 많은 데이터를 그냥 손 놓고 바라볼 수밖에 없었지만, PCA 는 '가장 정보가 많은 방향'을 수학적으로 콧 집어 주어 고차원의 안개를 걷어 준다.

1.1.1 직관 — 분산이 큰 방향을 찾는다

PCA 한 줄 요약 – 데이터가 가장 넓게 퍼진 방향(분산이 최대인 방향)을 첫 번째 새 축(제 1 주성분, PC1)으로 잡고, 그에 직교하면서 다음으로 넓게 퍼진 방향을 제 2 주성분(PC2)으로 잡는다. 이렇게 분산이 큰 순서대로 축을 골라 앞쪽 몇 개만 남기면, 정보 손실을 최소로 하면서 차원을 줄일 수 있다.

왜 하필 '분산이 큰 방향'일까? 분산은 데이터가 그 방향으로 얼마나 다양하게 퍼져 있는지를 나타낸다. 퍼짐이 큰 방향일수록 표본들을 서로 구별해 주는 정보를 많이 담고 있다. 반대로 모든 표본이 거의 같은 값을 갖는 방향, 즉 분산이 거의 0 인 방향은 버려도 정보 손실이 거의 없다. PCA 는 이 직관을 그대로 수학으로 옮긴 것이다.

아래 그림은 2 차원 데이터에 PCA 를 적용했을 때 찾아지는 두 축을 보여 준다. 빨간 화살표(PC1)는 데이터 구름이 가장 길게 늘어진 방향을, 초록 화살표(PC2)는 그에 수직이면서 두 번째로 길게 늘어진 방향을 가리킨다. 두 축은 항상 서로 직교한다.



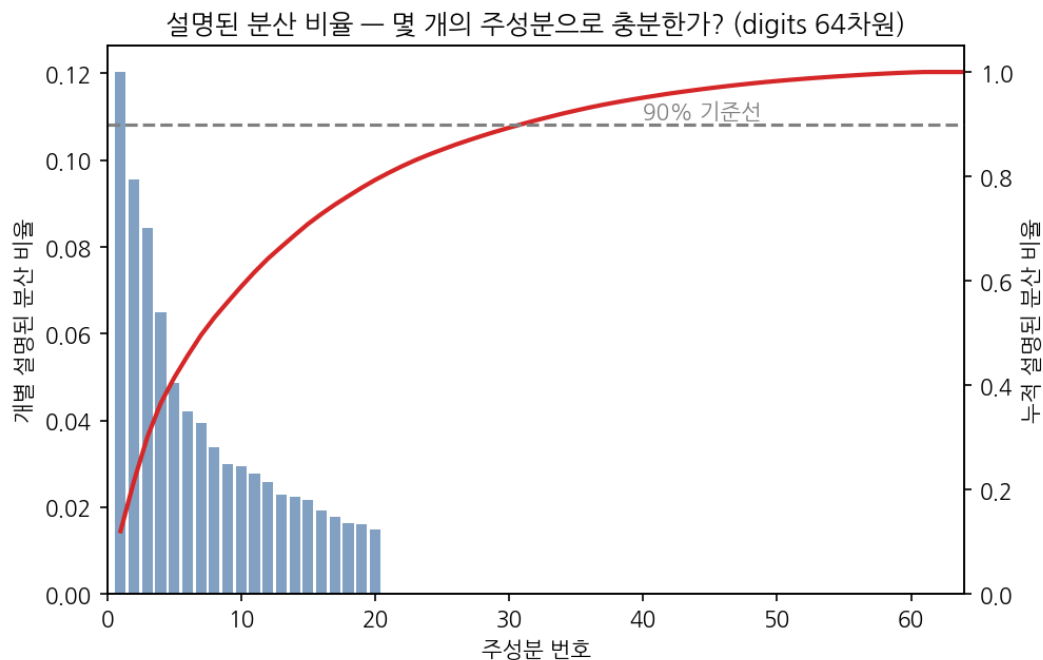
[그림 1-1] PCA 가 찾는 주성분 축 — PC1 은 분산 최대 방향, PC2 는 그에 직교

1.1.2 수식 개요 — 공분산 행렬과 고유분해

PCA의 계산은 의외로 깔끔하다. 먼저 각 변수의 평균을 0으로 맞추는 데이터 X 에 대해 공분산 행렬 $C = (1/n) X^T X$ 를 구한다. 공분산 행렬은 변수들이 서로 어떻게 함께 변하는지를 담고 있다. 이 행렬을 고유분해(eigen-decomposition)하면 고유벡터(eigenvector)와 고유값(eigenvalue)이 나온다.

핵심 대응 관계 - 고유벡터 = 새로운 축(주성분)의 방향, 고유값 = 그 축 방향의 분산 크기. 따라서 고유값이 큰 순서대로 고유벡터를 정렬해 위에서부터 k 개를 고르면, 분산을 가장 많이 보존하는 k 차원 부분 공간을 얻는다. '설명된 분산 비율(explained variance ratio)'은 (해당 고유값) ÷ (전체 고유값 합)으로, 그 주성분 하나가 전체 정보의 몇 %를 담는지를 알려 준다.

그렇다면 주성분을 몇 개나 남겨야 할까? 정답은 데이터마다 다르지만, 흔히 쓰는 기준은 '누적 설명된 분산 비율이 90% 또는 95%에 도달하는 지점까지'다. 아래 그림은 64차원 손글씨 숫자 데이터의 주성분별 분산 비율과 그 누적값이다. 막대(개별 비율)는 빠르게 작아지고, 빨간 누적선은 20~30개 주성분만으로 90% 선을 넘는다. 즉 64차원을 30차원 이하로 줄여도 정보의 90%가 살아남는다는 뜻이다.



[그림 1-2] 설명된 분산 비율과 누적 곡선 — 몇 개의 주성분이면 충분한가

1.1.3 scikit-learn 실습 — 붓꽃 데이터 2D 투영

이론을 충분히 봤으니 실제 코드로 확인해 보자. 4차원(꽃받침 길이·너비, 꽃잎 길이·너비)으로 기록된 붓꽃(iris) 데이터를 PCA로 2차원에 투영한다. PCA는 변수의 스케일에 민감하므로, 먼저 StandardScaler로 표준화한 뒤 적용하는 것이 정석이다. 표준화를 건너뛰면 단위가 큰 변수가 분산을 독차지해 결과가 왜곡된다.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.decomposition import PCA

iris = load_iris()
X, y = iris.data, iris.target          # X: (150, 4)

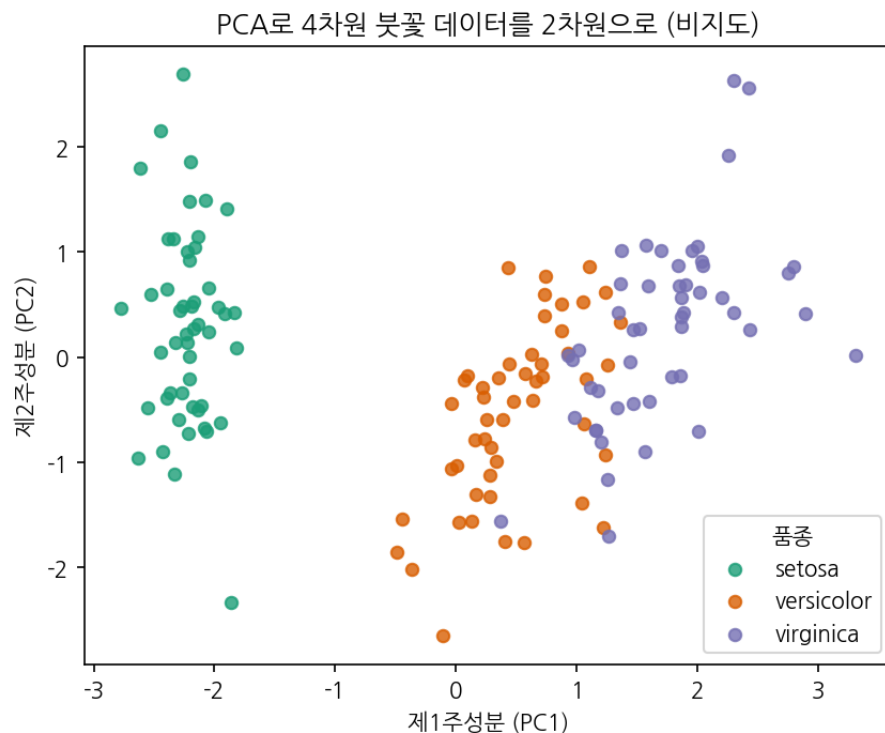
# 1) 표준화 - PCA 전에 반드시 스케일을 맞춘다
X_scaled = StandardScaler().fit_transform(X)

# 2) PCA 로 4 차원 -> 2 차원
pca = PCA(n_components=2)
Z = pca.fit_transform(X_scaled)        # Z: (150, 2)

print('설명된 분산 비율:', pca.explained_variance_ratio_)
print('누적:', pca.explained_variance_ratio_.sum())
# -> [0.7296 0.2285], 누적 0.958 (2 개 주성분이 정보의 95.8%를 보존)

# 3) 2D 산점도
colors = ['#1b9e77', '#d95f02', '#7570b3']
for k, name in enumerate(iris.target_names):
    m = y == k
    plt.scatter(Z[m, 0], Z[m, 1], c=colors[k], label=name, alpha=0.8)
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.legend()
plt.title('PCA: iris 4D -> 2D'); plt.show()

```



[그림 1-3] PCA 로 4 차원 붓꽃 데이터를 2 차원으로 투영한 결과(비지도)

결과를 보면 setosa(초록)는 나머지 두 품종과 완전히 분리되고, versicolor 와 virginica 는 약간 겹친다. 주목할 점은 PCA 가 품종 레이블 y 를 전혀 사용하지 않았다는 것이다. 오직 데이터의 분산 구조만으로 이만큼 분리된 그림을 얻었다. 이것이 비지도 차원 축소 힘이다.

흔한 실수 - PCA 전에 표준화를 빼먹는 것. 변수 단위가 제각각(예: 키 cm, 몸무게 kg, 소득 원)이면 값이 큰 변수가 분산을 독점해 주성분을 지배한다. 또 하나, fit 은 반드시 학습 데이터로만 하고 **테스트 데**

이터에는 transform 만 적용해야 한다. 테스트 데이터까지 fit 에 넣으면 정보 누설(data leakage)이 된다.

1.1.4 다양한 데이터셋 적용 예시 — diamonds · mpg · gapminder

붓꽃 한 가지만 보면 PCA 가 늘 깔끔하게 갈라준다는 착각에 빠지기 쉽다. 현실의 데이터는 변수 단위도 제각각이고, 군집이 겹치기도 한다. 그래서 성격이 다른 세 데이터셋(다이아몬드 가격 데이터, 자동차 연비 데이터, 국가별 사회·경제 데이터)에 같은 절차를 그대로 적용해 본다.

PCA 는 비지도학습이므로 라벨 없이 분산이 큰 방향을 찾고, 색은 단지 결과를 눈으로 확인하려고 입힌 것뿐이다.

공통 절차는 항상 같다 - ① 수치형 특징만 골라낸다 → ② StandardScaler 로 표준화한다 → ③ PCA(n_components=2)로 2 차원에 투영한다 → ④ explained_variance_ratio_로 정보 보존율을 확인한다. 데이터가 바뀌어도 이 네 단계는 변하지 않는다.

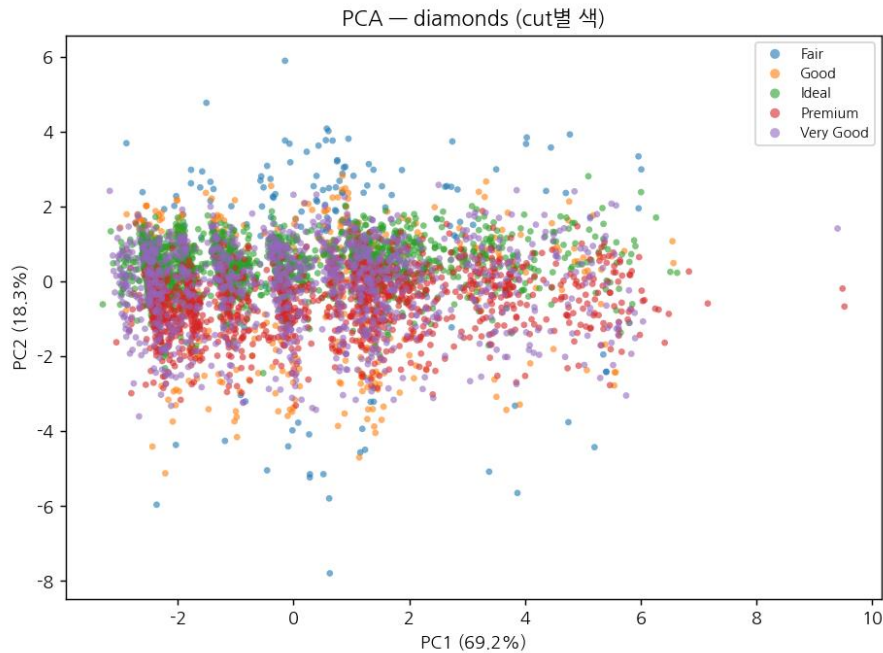
먼저 diamonds 다. 캐럿·깊이·테이블·가격·x·y·z 등 7 개 수치 특징을 쓰고, 5 천 개를 샘플링해 표준화한 뒤 PCA 를 돌린다. cut(5 등급)으로 색을 입혀 본다. 실행하면 PC1 이 약 69.2%, PC2 가 약 18.3%를 설명해 두 축만으로 전체 분산의 약 87.5%를 담는다. 크기 관련 변수(carat, x, y, z)가 강하게 상관되어 첫 축에 정보가 쏠린 결과다.

```
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'NanumGothic' # 한글 깨짐 방지(설치된 한글 폰트명으로)
plt.rcParams['axes.unicode_minus'] = False # 음수 부호 깨짐 방지

df = sns.load_dataset('diamonds').dropna()
df = df.sample(n=5000, random_state=42)
feats = ['carat', 'depth', 'table', 'price', 'x', 'y', 'z']
X = StandardScaler().fit_transform(df[feats])

pca = PCA(n_components=2, random_state=42)
emb = pca.fit_transform(X)
evr = pca.explained_variance_ratio_
print('설명분산비율:', evr)
# 실제 출력 -> [0.692 0.183] (두 축 합계 약 87.5%)

# --- 시각화 ---
cats = sorted(df['cut'].unique())
for c in cats:
    m = df['cut'].values == c
    plt.scatter(emb[m, 0], emb[m, 1], s=14, alpha=0.6, label=c)
plt.xlabel(f'PC1 ({evr[0]*100:.1f}%)') # 축 라벨에 설명분산비율 표기
plt.ylabel(f'PC2 ({evr[1]*100:.1f}%)')
plt.title('PCA - diamonds (cut 별 색)')
plt.legend()
plt.tight_layout()
plt.show()
```



[그림 1-4] PCA 로 본 diamonds(7 차원→2 차원), 색은 cut 등급

그림을 보면 cut 등급(색)은 거의 뒤섞여 있다. 당연하다 — PCA 는 cut 을 본 적이 없고, 오직 가격·크기 같은 변수의 분산만 좇았기 때문이다. 즉 '분산을 잘 보존한 축'이 '등급을 잘 가르는 축'과 일치한다는 보장은 없다.

두 번째는 mpg 다. 연비·실린더·배기량·마력·무게·가속 6 개 특징을 쓰고 결측치를 제거한다(마력에 6 개 결측). origin(usa/japan/europe)으로 색을 입힌다. 실행하면 PC1 이 약 79.8%로 분산 대부분을 설명하는데, 이는 '큰 차 ↔ 작은 차'라는 하나의 강한 축(배기량·무게·마력이 함께 커지고 연비는 작아지는)이 데이터를 지배하기 때문이다.

```
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

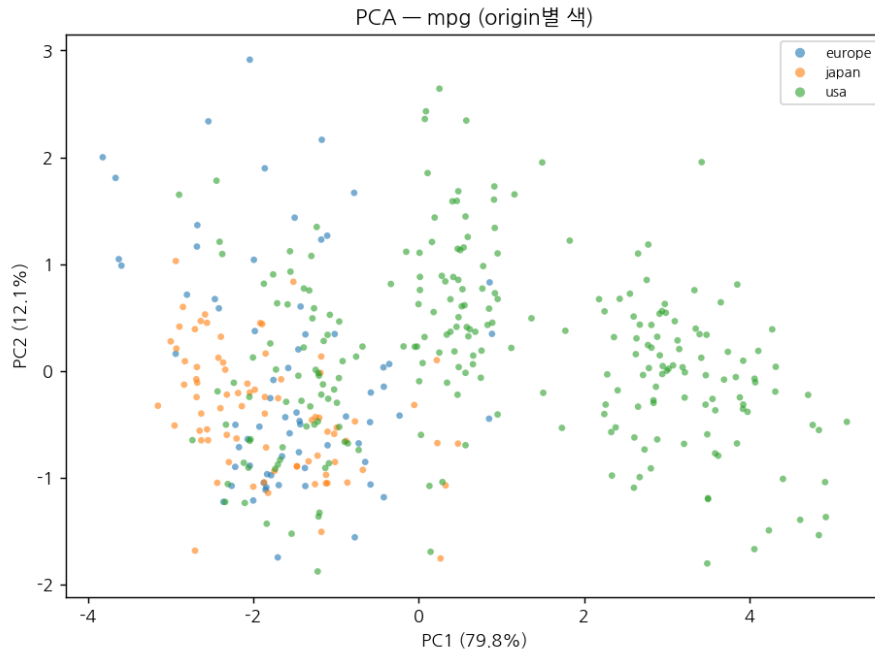
plt.rcParams['font.family'] = 'NanumGothic' # 한글 깨짐 방지(설치된 한글 폰트명으로)
plt.rcParams['axes.unicode_minus'] = False # 음수 부호 깨짐 방지

df = sns.load_dataset('mpg').dropna()
feats = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration']
X = StandardScaler().fit_transform(df[feats])

pca = PCA(n_components=2, random_state=42)
emb = pca.fit_transform(X)
evr = pca.explained_variance_ratio_
print(evr)
# 실제 출력 -> [0.798 0.121] (두 축 합계 약 92.0%)

# --- 시각화 ---
cats = sorted(df['origin'].unique()) # europe / japan / usa
for c in cats:
    m = df['origin'].values == c
    plt.scatter(emb[m, 0], emb[m, 1], s=18, alpha=0.7, label=c)
plt.xlabel(f'PC1 ({evr[0]*100:.1f}%)')
```

```
plt.ylabel(f'PC2 ({evr[1]*100:.1f}%)')
plt.title('PCA - mpg (origin 별 색)')
plt.legend()
plt.tight_layout()
plt.show()
```



[그림 1-5] PCA 로 본 mpg(6 차원→2 차원), 색은 origin

흥미롭게도 mpg에서는 PCA 만으로도 미국 차(usa)가 오른쪽으로 어느 정도 모인다. 미국 차가 대체로 크고 무거워서, '크기 축'인 PC1 이 우연히 원산지와 상관되기 때문이다. 이런 정렬은 항상 보장되는 것이 아니라 데이터의 특성에 따른 행운이다.

세 번째는 gapminder 다. seaborn에는 없어 plotly.express의 공개 데이터를 쓴다(plotly가 없으면 penguins로 대체해도 절차는 같다). 2007년 단면만 골라 기대수명·인구·1인당 GDP 3개 특징을 쓰되, 인구와 GDP는 분포가 한쪽으로 길어 로그 변환한 뒤 표준화한다. continent로 색을 입힌다. 실행하면 두 축이 약 93.9%를 설명하며, 대륙(특히 아프리카 vs 유럽)이 비교적 잘 갈린다 — 사회·경제 지표가 대륙별로 뚜렷이 다르기 때문이다.

```
import numpy as np, plotly.express as px
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'NanumGothic' # 한글 깨짐 방지(설치된 한글 폰트명으로)
plt.rcParams['axes.unicode_minus'] = False # 음수 부호 깨짐 방지

df = px.data.gapminder()
df = df[df['year'] == 2007].dropna().copy()
df['pop'] = np.log10(df['pop']) # 분포 보정
df['gdpPercap'] = np.log10(df['gdpPercap'])
feats = ['lifeExp', 'pop', 'gdpPercap']
X = StandardScaler().fit_transform(df[feats])

pca = PCA(n_components=2, random_state=42)
emb = pca.fit_transform(X)
```

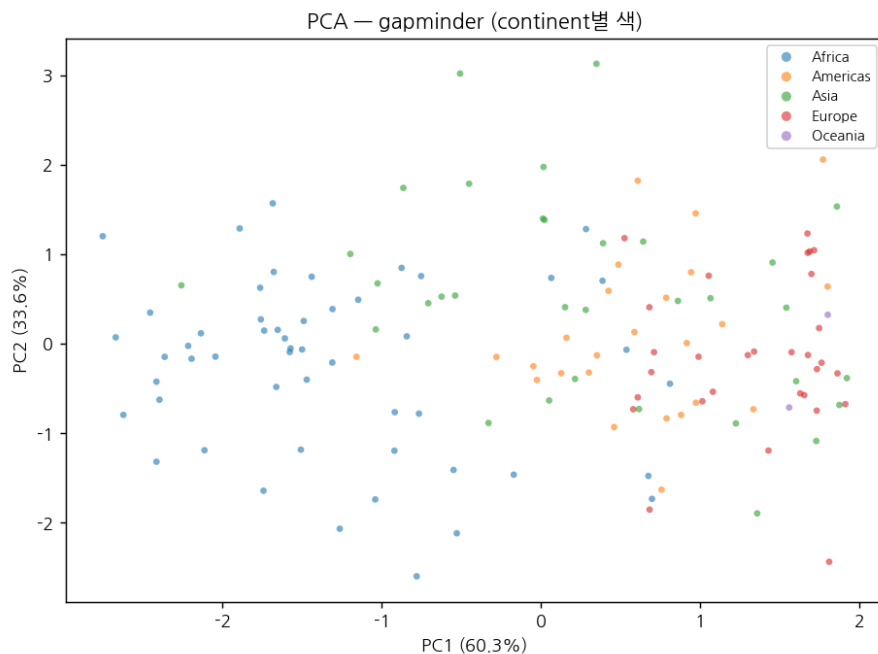


```

evr = pca.explained_variance_ratio_
print(evr)
# 실제 출력 -> [0.603 0.336] (두 축 합계 약 93.9%)

# --- 시각화 ---
cats = sorted(df['continent'].unique())
for c in cats:
    m = df['continent'].values == c
    plt.scatter(emb[m, 0], emb[m, 1], s=22, alpha=0.7, label=c)
plt.xlabel(f'PC1 ({evr[0]*100:.1f}%)')
plt.ylabel(f'PC2 ({evr[1]*100:.1f}%)')
plt.title('PCA - gapminder (continent 별 색)')
plt.legend()
plt.tight_layout()
plt.show()

```



[그림 1-6] PCA 로 본 gapminder 2007(3 차원→2 차원), 색은 continent

세 데이터셋의 교훈 - PCA 는 라벨을 모른 채 분산만 보존한다. 그래서 색이 잘 갈리면 그건 데이터가 그렇게 생긴 덕이지 PCA 가 분류를 한 것이 아니다. 라벨을 적극적으로 활용해 더 잘 가르고 싶다면 바로 다음 절의 LDA 가 필요하다.

1.2 LDA - 선형 판별 분석 (Linear Discriminant Analysis)

PCA 는 분산만 보고 축을 고르기 때문에, 정작 우리가 풀려는 분류 문제에는 별 도움이 안 되는 방향을 1 순위로 뽑을 때가 있다. '가장 넓게 퍼진 방향'이 반드시 '클래스를 가장 잘 가르는 방향'은 아니기 때문이다. 이 아쉬움에서 출발한 것이 LDA 다. LDA 는 레이블을 적극적으로 활용해 '클래스를 잘 구분하는 방향'을 직접 찾는다.

1.2.1 직관 — 클래스 간은 멀리, 클래스 안은 촘촘히

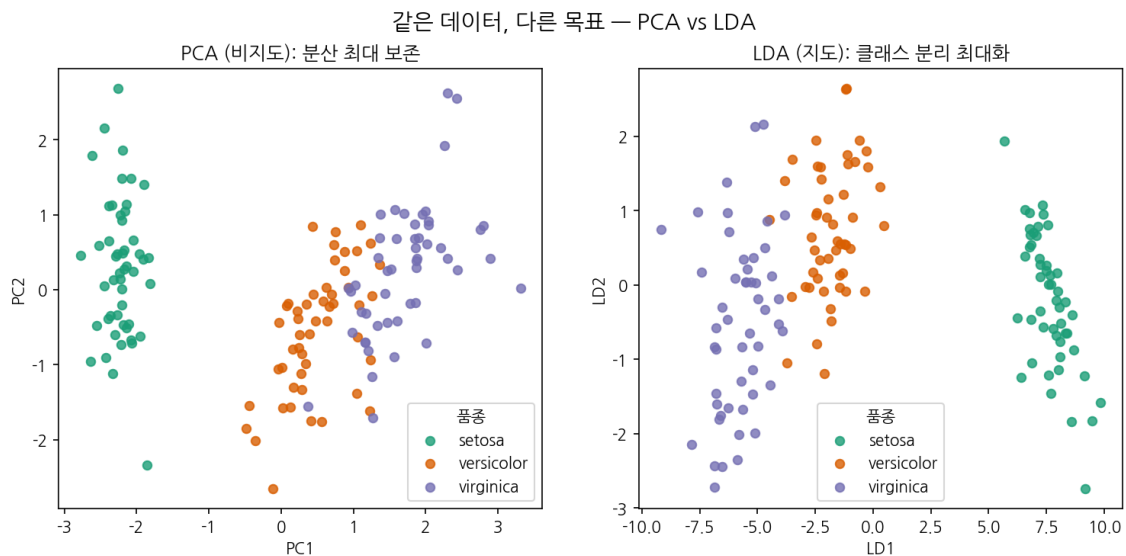
LDA 한 줄 요약 – 서로 다른 클래스의 중심(평균)은 최대한 멀어지게(클래스 간 분산 최대화), 같은 클래스 안의 점들은 최대한 모이게(클래스 내 분산 최소화) 만드는 축을 찾는다. 즉 '클래스 간 분산 ÷ 클래스 내 분산'을 최대로 하는 방향이 LDA의 새 축이다.

비유하자면 PCA는 '데이터가 어디로 가장 넓게 퍼졌나'만 보는 색맹 관찰자이고, LDA는 '어느 방향에서 봐야 팀이 가장 또렷하게 나뉘어 보이냐'를 따지는 심판이다. LDA는 각 표본이 어느 팀(클래스)인지 알기 때문에, 팀끼리 겹치지 않고 각 팀은 뽕뽕 뭉쳐 보이는 시점을 골라 준다.

한 가지 기억할 제약이 있다. LDA가 만들 수 있는 **새 축의 최대 개수**는 '클래스 수 - 1'이다. 붓꽃은 품종이 3개이므로 LDA 축은 최대 2개뿐이다. 클래스가 2개인 이진 분류라면 LDA는 단 하나의 축, 즉 1차원으로만 축소할 수 있다. PCA가 원본 차원까지 자유롭게 축을 만드는 것과 대비되는 지점이다.

1.2.2 PCA와의 차이 — 비지도 vs 지도

아래 그림은 똑같은 붓꽃 데이터를 PCA(왼쪽)와 LDA(오른쪽)로 각각 2차원에 내린 결과를 나란히 둔 것이다. 왼쪽 PCA는 레이블을 모른 채 분산만 보존했고, 오른쪽 LDA는 레이블을 써서 세 품종을 가능한 한 멀리 떼어 놓았다. 오른쪽에서 세 군집이 가로축(LD1)을 따라 훨씬 깔끔하게 분리된 것이 한눈에 보인다.



[그림 1-4] 같은 데이터, 다른 목표 — PCA(분산 보존) vs LDA(클래스 분리)

PCA vs LDA 정리 – 레이블이 없거나 데이터 구조 자체를 탐색하고 싶으면 PCA. **레이블이 있고** 분류 성능을 위해 차원을 줄이고 싶으면 LDA. 다만 클래스 수가 적으면 LDA의 축 개수도 그만큼 제한되니, 시각화를 위해 무조건 2차원이 필요하다면 이 제약을 미리 확인해야 한다.

1.2.3 scikit-learn 실습 — LDA로 차원 축소

코드는 PCA와 거의 같지만, 결정적 차이는 fit 단계에서 레이블 y를 함께 넘긴다는 점이다. 이 한 줄의 차이가 비지도와 지도를 가른다. LDA 역시 스케일에 민감하므로 표준화 후 적용한다.

```

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

iris = load_iris()
X, y = iris.data, iris.target
X_scaled = StandardScaler().fit_transform(X)

# 클래스 수가 3 개이므로 n_components 는 최대 2 (= 3 - 1)
lda = LinearDiscriminantAnalysis(n_components=2)
Z = lda.fit_transform(X_scaled, y)      # fit 에 레이블 y 를 함께 넘긴다!

# LDA 는 분류기로도 바로 쓸 수 있다
print('학습 데이터 정확도:', lda.score(X_scaled, y))    # -> 0.98

colors = ['#1b9e77', '#d95f02', '#7570b3']
for k, name in enumerate(iris.target_names):
    m = y == k
    plt.scatter(Z[m, 0], Z[m, 1], c=colors[k], label=name, alpha=0.8)
plt.xlabel('LD1'); plt.ylabel('LD2'); plt.legend()
plt.title('LDA: iris 4D -> 2D (supervised)'); plt.show()

```

LDA 는 차원 축소기이면서 동시에 분류기이기도 하다. fit_transform 으로 축소된 좌표를 얻을 수도 있고, score 나 predict 로 바로 분류 정확도를 잴 수도 있다. 붓꽃 데이터에서는 98% 정확도가 나온다. 차원을 줄이는 행위 자체가 곧 '클래스를 가르는 일'과 직결되기 때문에 생기는 특징이다.

흔한 실수 - n_components 를 클래스 수 이상으로 지정하는 것. 클래스가 3 개인데 n_components=3 을 주면 오류가 난다. 또한 LDA 는 각 클래스가 대략 정규분포이고 공분산이 비슷하다고 가정한다. 이 가정이 크게 깨지면 LDA 의 분리력이 떨어지니, 그럴 때는 PCA 나 비선형 기법을 함께 검토하자.

1.2.4 다양한 데이터셋 적용 예시 — diamonds · mpg · gapminder

PCA 는 라벨을 무시했지만 LDA 는 정반대다. '클래스를 가장 잘 가르는 방향'을 라벨 y 를 보고 직접 찾는다. 같은 세 데이터셋에 LDA 를 적용해, 앞 절 PCA 그림과 나란히 비교해 보면 차이가 한눈에 보인다. LDA 는 지도학습이므로 fit 단계에 반드시 범주형 타깃을 함께 넘긴다.

핵심 제약 – LDA 가 만들 수 있는 축은 최대 (클래스 수 - 1)개다. 클래스가 3 개면 2 개, 5 개면 4 개까지 가능하다. 그래서 클래스 2 개짜리 데이터는 1 차원으로만 줄어든다. 아래 세 예시는 모두 클래스가 3 개 이상이라 2 차원 산점도를 그릴 수 있다.

diamonds 의 cut 은 5 개 클래스라 LD 축을 2 개 뽑을 수 있다. PCA 코드와 거의 같지만, 결정적 차이는 fit_transform(X, y)에 **라벨 y** 가 들어간다는 점이다. 이 한 줄이 비지도와 지도를 가른다.

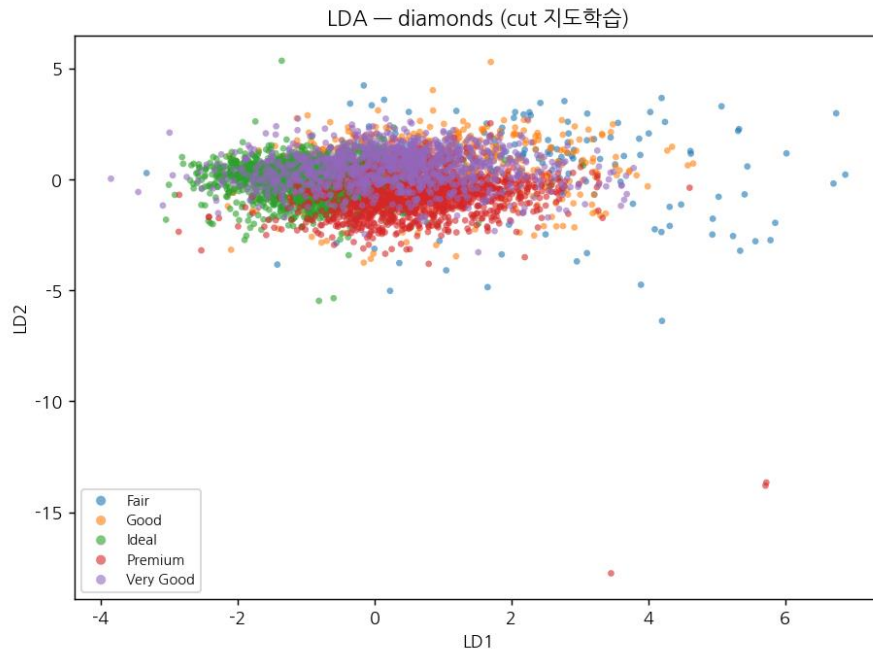
```
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import matplotlib.pyplot as plt

plt.rcParams['font.family'] = 'NanumGothic' # 한글 깨짐 방지(설치된 한글 폰트명으로)
plt.rcParams['axes.unicode_minus'] = False # 음수 부호 깨짐 방지

df = sns.load_dataset('diamonds').dropna().sample(n=5000, random_state=42)
feats = ['carat', 'depth', 'table', 'price', 'x', 'y', 'z']
X = StandardScaler().fit_transform(df[feats])
y = LabelEncoder().fit_transform(df['cut']) # 라벨이 필수!

lda = LinearDiscriminantAnalysis(n_components=2)
emb = lda.fit_transform(X, y) # PCA 와 달리 y 를 함께 넘긴다

# --- 시각화 ---
cats = sorted(df['cut'].unique())
for c in cats: # cut 등급별로 색 구분
    m = df['cut'].values == c
    plt.scatter(emb[m, 0], emb[m, 1], s=14, alpha=0.6, label=c)
plt.xlabel('LD1') # LDA 축은 판별축(LD1, LD2)
plt.ylabel('LD2')
plt.title('LDA - diamonds (cut 지도학습)')
plt.legend()
plt.tight_layout()
plt.show()
```



[그림 1-7] LDA 로 본 diamonds, 색은 cut(지도학습)

그림 1-4(PCA)와 비교하면, 등급이 완벽히 갈리진 않아도 PCA 보다 색의 띠가 한 방향으로 더 늘어선다. cut 등급은 본래 가격·크기만으로 깔끔히 결정되는 값이 아니라(컷의 품질은 측정 변수에 약하게만 묻어난다) 분리에 한계가 있지만, 그래도 라벨을 본 LDA 가 PCA 보다 등급 방향을 더 뚜렷이 살린다.

mpg 의 origin 은 3 개 클래스다. LDA 로 줄이면 차이가 극적으로 드러난다. 그림 1-5(PCA)에서 미국 차가 '어느 정도' 모였다면, LDA 에서는 LD1 축을 따라 미국 차가 확실하게 한쪽으로 분리된다.

```
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import matplotlib.pyplot as plt

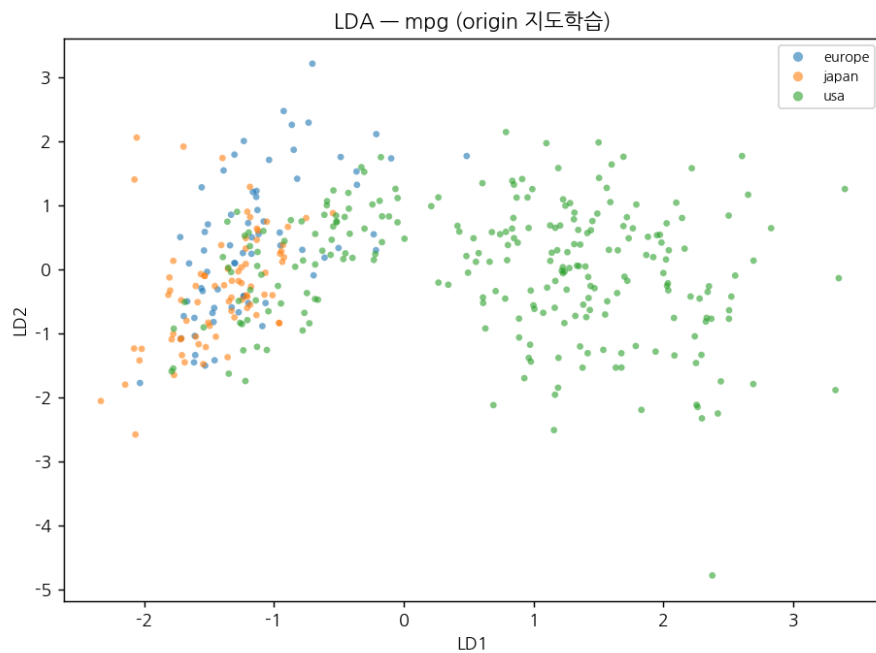
plt.rcParams['font.family'] = 'NanumGothic' # 한글 깨짐 방지(설치된 한글 폰트명으로)
plt.rcParams['axes.unicode_minus'] = False # 음수 부호 깨짐 방지

df = sns.load_dataset('mpg').dropna()
feats = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration']
X = StandardScaler().fit_transform(df[feats])
y = LabelEncoder().fit_transform(df['origin'])

lda = LinearDiscriminantAnalysis(n_components=2) # 클래스 3 개 -> 최대 2
emb = lda.fit_transform(X, y)

# --- 시각화 ---
cats = sorted(df['origin'].unique())
for c in cats:
    m = df['origin'].values == c
    plt.scatter(emb[m, 0], emb[m, 1], s=18, alpha=0.7, label=c)
plt.xlabel('LD1')
plt.ylabel('LD2')
plt.title('LDA — mpg (origin 지도학습)')
```

```
plt.legend()
plt.tight_layout()
plt.show()
```



[그림 1-8] LDA 로 본 mpg, 색은 origin(지도학습) — usa 가 LD1 으로 또렷이 분리

이것이 LDA 가 '판별' 분석인 이유다. 같은 특징·같은 표준화인데도 라벨을 쓴 것만으로 미국/비미국 경계가 선명해졌다. 유럽과 일본 차는 서로 닮아 여전히 겹치지만, 이는 두 지역 차의 물리적 특성이 실제로 비슷하기 때문이다.

gapminder 의 continent 는 5 개 클래스다. 특징이 3 개뿐이라 LDA 가 뽑을 수 있는 축은 $\min(2, 5-1)=2$ 개다. 대륙별 사회·경제 격차가 워낙 커서, LDA 는 아프리카·유럽·아시아 무리를 PCA 보다 더 단정하게 떼어 놓는다.

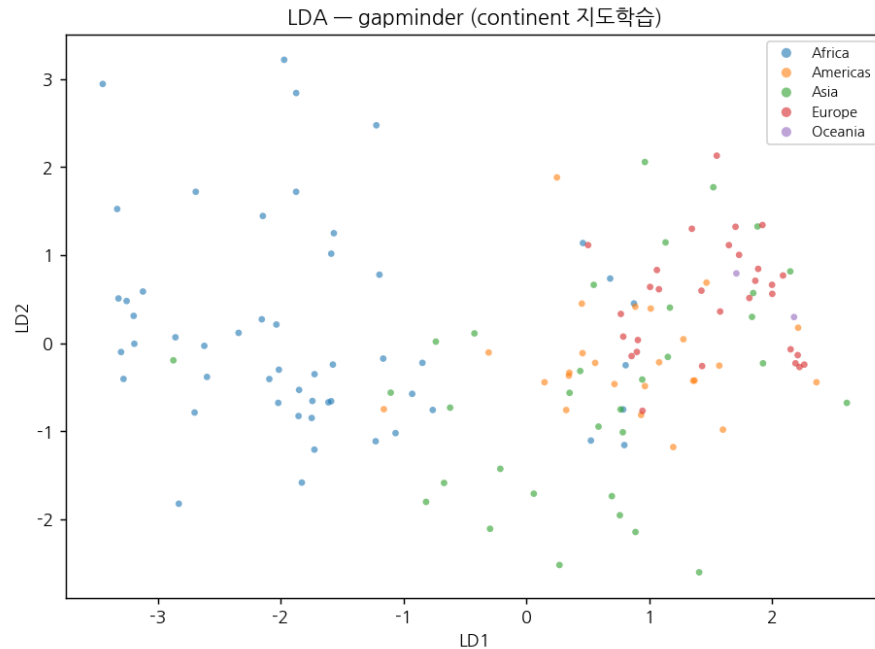
```
import numpy as np, plotly.express as px
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import matplotlib.pyplot as plt
plt.rcParams['font.family'] = 'NanumGothic' # 한글 깨짐 방지(설치된 한글 폰트명으로)
plt.rcParams['axes.unicode_minus'] = False # 음수 부호 깨짐 방지

df = px.data.gapminder()
df = df[df['year'] == 2007].dropna().copy()
df['pop'] = np.log10(df['pop']); df['gdpPercap'] = np.log10(df['gdpPercap'])
feats = ['lifeExp', 'pop', 'gdpPercap']
X = StandardScaler().fit_transform(df[feats])
y = LabelEncoder().fit_transform(df['continent'])

lda = LinearDiscriminantAnalysis(n_components=2)
emb = lda.fit_transform(X, y)

# --- 시각화 ---
cats = sorted(df['continent'].unique())
for c in cats:
    m = df['continent'].values == c
```

```
plt.scatter(emb[m, 0], emb[m, 1], s=22, alpha=0.7, label=c)
plt.xlabel('LD1')
plt.ylabel('LD2')
plt.title('LDA - gapminder (continent 지도학습)')
plt.legend()
plt.tight_layout()
plt.show()
```



[그림 1-9] LDA 로 본 gapminder 2007, 색은 continent(지도학습)

PCA vs LDA 한 줄 정리 – 같은 데이터, 같은 표준화여도 '라벨을 보느냐'가 분리력을 가른다. 다만 LDA 는 클래스 안/밖 분산비를 최대화하는 선형 변환이라, 클래스 경계가 곡선이면 한계가 있다. 그럴 때는 비선형 기법인 t-SNE 를 본다.

3. 정리 및 요약

핵심 개념 – PCA 는 분산을 최대로 보존하는 비지도 선형 기법으로 압축·전처리에 강하다. LDA 는 레이블을 써서 클래스 분리를 최대화하는 지도 선형 기법으로 분류 전 차원 축소에 좋다(축은 클래스수-1 개)

기법은 경쟁 관계가 아니라 상황에 따라 골라 쓰는 도구다. 변수를 줄여 모델에 넣고 싶고 레이블이 없다면 PCA, 레이블이 있고 분류 성능이 목적이려면 LDA 를 쓰면 된다.

- 레이블이 있는가? — 있으면 LDA 를 고려, 없으면 PCA
- 새 데이터에 변환을 재적용해야 하는가? — 그렇다면 PCA·LDA(t-SNE 는 부적합).
- 어떤 기법이든 스케일에 민감하므로 표준화를 먼저 한다.

4. 과제

문제 1. 차원의 저주란 무엇이며, 차원이 늘어날 때 거리 기반 알고리즘이 어려움을 겪는 이유를 설명하라.

문제 2. PCA 에서 공분산 행렬의 고유값과 고유벡터는 각각 무엇을 의미하는가? '설명된 분산 비율'은 어떻게 계산하는가?

문제 3. PCA 와 LDA 의 가장 근본적인 차이를 '레이블 사용'과 '최적화 목표' 두 측면에서 비교하라. 또 클래스가 4 개일 때 LDA 가 만들 수 있는 최대 축 개수는?

5. 과제 해답

해답 1. 차원이 늘수록 같은 표본 수로 채워야 할 공간이 기하급수적으로 커져 데이터가 희박해진다. 그 결과 점들 사이 거리가 모두 비슷해져 '가까운 이웃'과 '먼 점'의 구분이 흐려지고, KNN·군집화 등 거리·밀도 기반 알고리즘의 성능이 급격히 떨어진다.

해답 2. 고유벡터는 새로운 축(주성분)의 방향, 고유값은 그 축 방향의 분산 크기다. 설명된 분산 비율은 (해당 고유값) ÷ (전체 고유값 합)으로, 그 주성분이 전체 정보의 몇 %를 담는지를 나타낸다.

해답 3. PCA 는 레이블을 쓰지 않는 비지도 기법으로 '분산 최대 보존'이 목표이고, LDA 는 레이블을 쓰는 지도 기법으로 '클래스 간 분산 ÷ 클래스 내 분산 최대화'가 목표다. 클래스가 4 개면 LDA 의 최대 축 개수는 $4-1 = 3$ 개다.